

TPFI 2025/26

Hw 5: A taste of C

assegnato: 22 maggio 2026, consegna 9 giugno 2026

1. Type Safety

L'esistenza del tipo `void*` è il più evidente segno del fatto che in C sia un linguaggio non type-safe, nonostante la buona volontà dei compilatori moderni che hanno proibito diverse possibilità presenti nel C standard delle origini.

Scrivere un programma C che dà risultati diversi se compilato su un'architettura big-endian (il primo byte è il più significativo, tipico di Motorola, Sun, IBM...) oppure su una little-endian (il primo byte è il meno significativo, tipico di Intel e Digital...).

Per chi disponesse di un Mac con processori di famiglia M, potrebbe farmi sapere se sono little o big-endian. Io non lo so ☺.

2. Array Mutabili, mergeSort adattivo

Tradizionalmente l'algoritmo di ordinamento *mergeSort* viene presentato in forma ricorsiva.

2.1 Osservando che *mergeSort*, di fatto, fa sempre le stesse cose, non è difficile scrivere una versione “genuinamente” iterativa (che non è quindi la traduzione della versione ricorsiva con gestione esplicita di una pila) che fonde (ad albero) le sequenze ordinate a partire dai segmenti di lunghezza al più 1 che sono ovviamente già ordinati. È molto semplice scriverlo assumendo la lunghezza del vettore una potenza di 2, ma con un po' di tecnica si riesce facilmente a risolvere il caso generale. Scrivete tale programma in C.

2.2 Usualmente si dice anche che *mergeSort* non trae vantaggio da situazioni favorevoli e, infatti, non ho mai visto nei libri una versione “adattiva” di *mergeSort* (analoga a quella richiesta nell'Homework 2) che comincia le operazioni di fusione a partire dalle sequenze già ordinate e risulta quindi essere di complessità lineare se il vettore è già ordinato e in generale $\Theta(n \log k)$ dove k è il numero delle sequenze ordinate presenti nel vettore (al più k è n).

Scrivete la versione adattiva di *mergeSort* cercando di memorizzare la minima informazione necessaria allo scopo.

3. Quello che in Haskell non si può fare I: alberi con pointer up

Considerate gli alberi binari in cui ciascun nodo ha un puntatore al(l'unico) padre. Tale pointer (diciamo *up*) deve soddisfare alla proprietà $x \rightarrow left \rightarrow up = x$ e $x \rightarrow right \rightarrow up = x$, per ogni nodo x che non sia una foglia, e $x \rightarrow up \rightarrow left = x$ e $x \rightarrow up \rightarrow right = x$ rispettivamente per tutti i nodi che sono figli sinistri e figli destri. Infine $x \rightarrow up = \text{NULL}$ se x è la radice.

Scrivere un programma **C** che fa una visita in profondità di un siffatto albero (ad esempio una in-order) *completamente iterativa e senza strutture dati ausiliarie*.

4. Quello che in Haskell non si può fare II: Crivello di Eulero

Scrivere una funzione **C** che implementi il *crivello di Eulero*. Non è ovvio “saltare” in modo efficiente i numeri già cancellati per trarne vantaggio nelle successive cancellazioni. La soluzione che vi propongo di implementare consiste nell’usare un vettore di coppie di naturali, *succ* e *prec*, come una *lista doppiamente concatenata* in cui nella posizione i , se i non è stato cancellato, *succ* è il numero di posizioni che occorre saltare per andare al prossimo numero non cancellato, mentre *prec* è il numero di posizioni che occorre saltare (all’indietro) per andare al precedente numero non cancellato.

Potete ad esempio definire un tipo **Pair** che è una coppia di interi *succ* e *prec* e definiamo un vettore di **Pair**. Questo vettore va inizializzato con tutti 1 (che significa appunto che tutti i numeri sono ancora potenziali primi). Quindi lo stato del vettore, inizialmente è il seguente (dove # significa ‘non rilevante’):

<i>pos</i>	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
<i>succ</i>	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
<i>prec</i>	#	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Dopo aver cancellato i multipli di due, il vettore avrà i seguenti valori:

<i>pos</i>	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
<i>succ</i>	1	2	#	2	#	2	#	2	#	2	#	2	#	2	#	2	#	2	#	2	#	2	#
<i>prec</i>	#	1	#	2	#	2	#	2	#	2	#	2	#	2	#	2	#	2	#	2	#	2	#

Ora, partendo da 3 posso facilmente saltare sui numeri non cancellati. Moltiplicando questi per 3 ottengo quelli da cancellare in questa iterazione, e cioè 9, 15, 21, ottenendo la seguente situazione:

<i>pos</i>	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
<i>succ</i>	1	2	#	2	#	4	#	#	#	2	#	4	#	#	#	2	#	4	#	#	#	2	#
<i>prec</i>	#	1	#	2	#	2	#	#	#	4	#	2	#	#	#	4	#	2	#	#	#	4	#

Nell’esempio, a questo punto ho finito, perché il prossimo numero non cancellato è il 5 e $5^2 > 24$. Partendo da 2 e scorrendo il vettore usando i puntatori *succ* posso stampare tutti i numeri non cancellati che sono a questo punto necessariamente primi (scrivere una funzione **printPrimes** allo scopo).

Voi dovete scrivere una funzione: `Pair* eulerSieve(int n);` che restituisce un vettore di coppie da cui sia possibile ricostruire tutti i numeri primi da 2 a n .

OSSERVAZIONI: I puntatori *prev* servono essenzialmente per effettuare in modo efficiente le operazioni di cancellazione. Le cancellazioni sono ‘problematiche’ perché sono operazioni distruttive sulla struttura dati e potrebbero, se fatte con poca cura, rendere inconsistente lo stato del vettore.

SPERIMENTAZIONI: Verificare che questo programma risulta effettivamente più efficiente del crivello di Eratostene. Ovviamente, il guadagno asintotico è modesto ($\Theta(n)$ invece che $\Theta(n \ln \ln n)$) e fa operazioni più complicate. Occorrerà provarlo per un qualche n sufficientemente grande (ordine di migliaia o milioni...).